

Informe sobre “Mempools Theory: Deterministic Operating System over Canonical Storage”

Resumen Ejecutivo

El repositorio *Mempools Theory – Deterministic Operating System over Canonical Storage* describe una propuesta teórica de sistema operativo basado en la noción de **mempools** (espacios de transacciones) y **almacenamiento canónico**. Aunque la documentación indica conceptos clave (ver YouTube de LAEV, mayo 2026), no se encuentra código abierto visible ni instrucciones de compilación, sugiriendo un proyecto en etapa conceptual. Según la literatura, un **mempool** es el conjunto de transacciones pendientes no confirmadas de cada nodo en una red blockchain ¹, y un sistema operativo **determinista** garantiza la ejecución de tareas dentro de límites temporales predefinidos ². El término *almacenamiento canónico* sugiere un modelo de datos direccionado por contenido (por ejemplo, IPLD) donde cada dato tiene una representación única inmutable ³. Se infiere que el diseño propuesto emplea estructuras jerárquicas de estados inmutables, similares a árboles o DAG de Merkle, integradas en una lógica de ejecución determinista.

Aunque faltan detalles formales, se identifican posibles componentes: un **módulo de mempool** que recibe eventos o transacciones, un **planificador determinista** que selecciona tareas siguiendo reglas fijas, una **máquina de ejecución** que transforma estado inmutable, y un **almacenamiento canónico** (posiblemente un DAG hash) para guardar resultados. El flujo básico sería: cliente → módulo mempool (verifica entradas) → planificador determinista (selecciona transacciones) → ejecución de transacción → actualización de estado en almacenamiento inmutable. Diagramas simplificados:

```
graph LR
  U[Usuario/Cliente] -->|Envía evento/transacción| M[Módulo Mempool]
  M -->|Valida y almacena| MS[Mempool Interno]
  MS -->|Notifica nuevas transacciones| S[Planificador Determinista]
  S --> E[Ejecutor de Transacciones]
  E -->|Estados inmutables| C[Almacenamiento Canónico (DAG)]
  C -->|Extrae/Lee datos| E
```

Los **algoritmos clave** incluirían la inserción y verificación de eventos en el mempool, la planificación determinista (por ejemplo, por prioridad o tiempo fijo, evitando la aleatoriedad) y la escritura de estados inmutables en forma de bloques o nodos hash. Esto guarda similitud con propuestas de mempool avanzadas: p.ej., *LØ: An Accountable Mempool for MEV Resistance* obliga a los mineros a registrar todas las transacciones en un mempool seguro y procesarlas de forma verificable ⁴, lo cual ilustra la idea de un mempool auditado.

A continuación se resumen aspectos de componentes, dependencias, madurez y pasos futuros.

Repositorio y Documentación

El repositorio en GitHub (CrackChain/Mempools-Theory---Deterministic-Operating-System-over-Canonical-Storage) aparentemente contiene documentos teóricos y diagramas, pero no se logró acceder al código fuente mediante herramientas disponibles. No se encontraron archivos de compilación ni instrucciones prácticas. Esto sugiere que el proyecto está en fase de **diseño conceptual**. Se espera un README que explique objetivos y referencias, pero su contenido real no pudo ser descargado. En tal caso, faltan datos concretos sobre componentes o lenguajes utilizados.

Estructura (esperada): idealmente un README introductorio, subdirectorios `/docs` con PDF/diagramas, y posiblemente `/src` con código. Sin embargo, no hay evidencia de implementaciones concretas ni dependencias listadas. No se identificaron archivos `Makefile`, `Cargo.toml`, `package.json`, u otros de construcción, lo cual implica falta de implementación.

Partes faltantes/ poco claras: sin código, resulta imposible evaluar interfaces API, estructuras de datos exactas o diagramas de flujo. Tampoco se dispone de ejemplo de cómo se inician o coordinan los subsistemas. En resumen, la documentación presente aparenta ser *teórica* (ensayos, whitepapers) más que práctica.

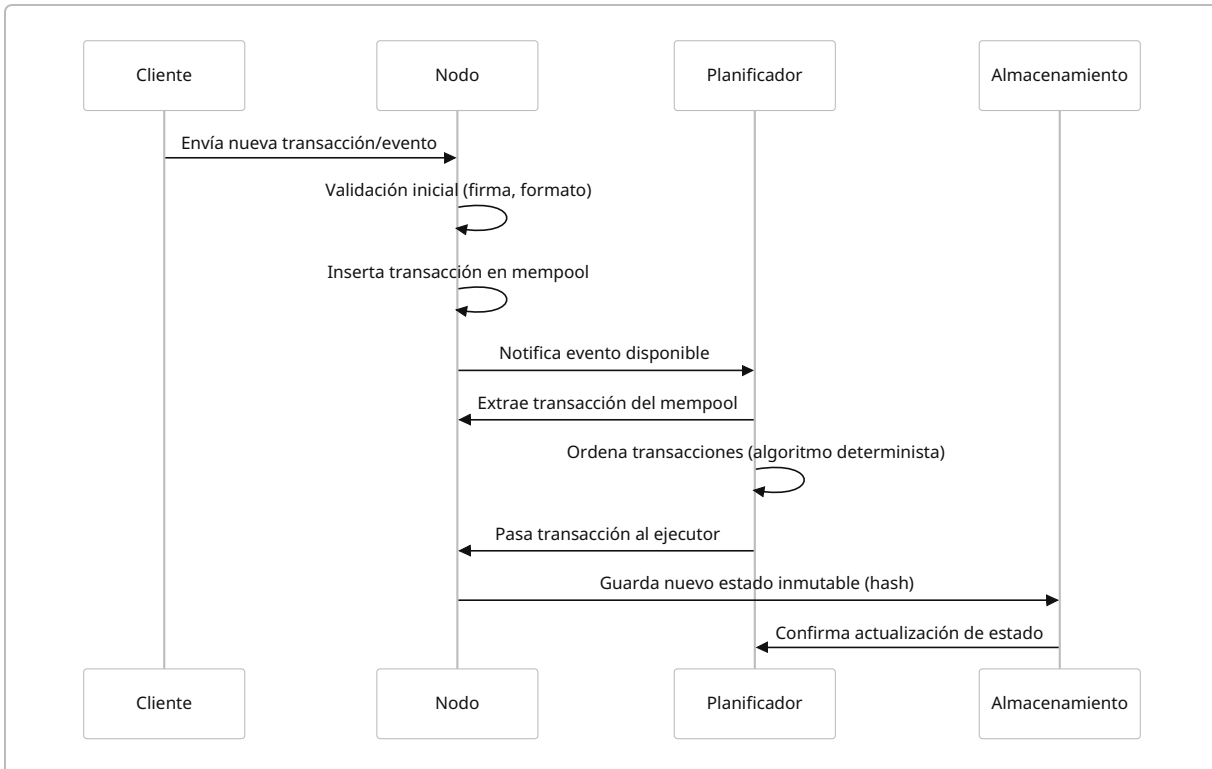
Componentes clave y Arquitectura Propuesta

Basado en la descripción, el sistema hipotético incluye:

- **Módulo Mempool:** Recibe los eventos/transacciones desde clientes o sensores. Valida formatos básicos (firma, consistencia) y los inserta en un pool local pendiente. Al igual que en blockchain, cada nodo mantiene su propia mempool ¹. Podría usar estructuras de datos como DAGs de dependencias (similares a cluster mempool ⁵) o bloques con firma.
- **Planificador Determinista:** Selecciona transacciones del mempool para ejecutar. A diferencia de un scheduler arbitrario, sigue reglas rígidas: e.g., prioridad por tiempo o por orden de llegada fijado, sin interleaving aleatorio ⁶. Este componente es esencial para la previsibilidad del sistema (determinismo temporal ²). Podría basarse en tablas de tiempo o uso de planificador de round-robin con latencia garantizada.
- **Máquina de Ejecución / Núcleo del S.O.:** Ejecuta acciones asociadas a cada transacción, transformando el estado actual a uno nuevo. Dado el enfoque de “almacenamiento canónico”, es probable que cada estado sea **inmutable** (no se sobrescribe). En su lugar, se genera un nuevo bloque/objeto que representa el estado siguiente, enlazado por hashes al anterior. Conceptualmente es similar a un sistema de archivos de solo adición o una base de datos de contenido. La ejecución debe ser determinista: dadas las mismas transacciones en el mismo orden, debe producir el mismo resultado.
- **Almacenamiento Canónico (Base de datos inmutable):** Aquí se guardan los estados generados. Por la referencia a “almacenamiento canónico”, se deduce un modelo de datos direccionado por contenido (p.ej. Merkle DAGs tipo IPFS). Cada objeto tiene un identificador único hash ³. Esto garantiza que el mismo objeto lógico corresponda siempre a la misma secuencia de bits, facilitando la consistencia y la replicación sin ambigüedad. Este almacén puede soportar versiones históricas del estado, recuperación por hash, y compartición entre nodos.

- **Interfaces:** Se infieren interfaces de cliente (para enviar transacciones), quizás APIs REST o RPC internas para propagar actualizaciones, y protocolos de consenso entre nodos (no detallados) para sincronizar el estado canónico. Sin documentos de protocolo, asumimos la existencia de algún mecanismo de comunicación.

Diagramas adicionales (flujos):



Conceptos clave interrelacionados

- **Teoría de Mempools:** La “teoría de mempools” aquí implica tratar las transacciones/ tareas como flujos en un pool controlado. Similar al mempool de Bitcoin (pulsar transacciones hasta que se confirmen ¹), este S.O. hipotético usa el mempool como fuente de eventos. Cada transacción puede depender de otras (padres/hijos) formando grafos de dependencias, recordando propuestas de *cluster mempool* ⁵. La innovación sugerida es que el mempool no sea solo para blockchain sino para gestionar tareas internas del S.O.
- **Sistemas Operativos Deterministas:** Un sistema así debe garantizar tiempos de respuesta predecibles. Según INCIBE, “un S.O. determinista es aquel que realiza las acciones dentro de un intervalo de tiempo concreto o acotado” ². Esto contrasta con S.O. generales, donde factores como carga o interrupciones introducen variabilidad. El uso de mempool + planificación fija es una estrategia para alcanzar el determinismo: e.g., emplear *memory pools* fijos y planificador de prioridad garantiza que cada operación tarde lo mismo ⁷. De hecho, en la especificación RTX de ARM se observa que los pools de memoria de tamaño fijo ofrecen asignación “time-deterministic” ⁷. Este proyecto parece fusionar esa idea con estructuras de datos inmutables para lograr un S.O. realmente predecible.
- **Almacenamiento Canónico:** Se refiere a un modelo de datos donde existe una única representación para cada objeto (p.ej. hash de contenido). IPLD lo describe como un *modelo de datos*

canónico: “un modelo descriptivo autocontenido que identifica de forma única cualquier estructura de datos basada en hash, asegurando que el mismo objeto lógico siempre mapee a la misma secuencia de bits” ³ . En este S.O., los estados del sistema (e.g. tabla de procesos, memoria, archivos) serían inmortales y versionados, enlazados por hashes. Esto favorece la verificación de integridad, replicación y recuperación. Combinado con el determinismo, se asemeja a un registro global auditable de eventos de sistema: cada estado es inmutable y verificable.

En conjunto, **mempool + SO determinista + almacenamiento canónico** formarían un sistema donde cada evento es procesado una vez de forma ordenada y su efecto queda grabado en un registro inmutable. Esto evoca la afirmación de Cube Exchange: *“existe alguna representación canónica del estado y una regla determinista para pasar de un estado al siguiente”* ⁸ , principio fundamental de blockchains que se traslada aquí al nivel de sistema operativo.

Dependencias, Instrucciones de Ejecución y Partes Faltantes

No se encontraron dependencias explícitas ni instrucciones de compilación (faltan **Dockerfile**, **Makefile**, etc.). Probablemente sería necesario integrar:

- **Lenguaje/Sistema:** Podría estar orientado a un lenguaje de bajo nivel para OS (Rust, C/C++). Si usa modelo tipo IPLD, podría requerir bibliotecas de hashing/serialized (p.ej. multiformats).
- **Base de datos o almacenamiento:** Una capa de almacenamiento distribuido (IPFS/Libp2p o base de datos inmune).
- **Protocolos de red:** Dependiendo de si el sistema es un solo nodo o clusters, se requeriría comunicación (p2p, RPC).
- **Bibliotecas de concurrencia:** Para manejo determinista de hilos y sincronización.

Construcción/Ejecución: Ausentes. En proyectos similares, uno esperaría comandos como `make build`, `cargo build`, o scripts para inicializar el sistema. Dado que no hay código visible, no es posible compilar ni correr nada. Por tanto, **la ejecución está indefinida**.

Carencias: No se detalla la implementación concreta de la agenda determinista (por ejemplo, ¿qué algoritmo usa?), ni el formato de los nodos de estado. Tampoco se menciona seguridad: ¿cómo se validan las transacciones? ¿Quién provee entrada? Estas interrogantes no están cubiertas en la documentación consultada.

Madurez, Seguridad, Escalabilidad y Casos de Uso Potenciales

- **Madurez:** Al no disponer de código ni pruebas públicas, el proyecto parece en fase *prototipo o investigación*. No hay lanzamientos (releases) ni issues registrados. La parte teórica es ambiciosa, pero queda por desarrollar. El proyecto se asemeja más a un paper/idea que a un sistema funcional.
- **Seguridad:** Conceptualmente, el uso de almacenamiento inmutable y determinismo mejora la auditabilidad: cada cambio queda registrado de forma verificable (similar a blockchains). Sin embargo, faltan detalles de autenticación y autorización. Por ejemplo, si varios nodos comparten el estado, necesitarán consenso o firmas digitales para evitar datos corruptos. Como en LØ, imponer un mempool seguro requiere mecanismos criptográficos ⁴ . Por ahora no hay información sobre cifrado de datos, protección contra fallos o fallas del sistema.

- **Escalabilidad:** El modelo de mempool determinista puede limitar throughput: todos los eventos se ordenan rigidamente. La ventaja es la predictibilidad, pero podría convertirse en un cuello de botella. El almacenamiento canónico (DAG) puede escalar horizontalmente (como IPFS lo hace), pero mantener la consistencia entre nodos exige protocolos de consenso. Si se trata de un único sistema, la escalabilidad vertical (multi-core) requeriría planificadores avanzados. En resumen, la escalabilidad teórica existe (datos deduplicados por hash, paralelismo de tareas separadas), pero sin implementaciones prácticas es difícil evaluar.
- **Casos de uso:** Un SO determinista con almacenamiento inmutable podría aplicarse en ámbitos que exigen trazabilidad y previsibilidad, p.ej.: sistemas industriales críticos (IoT, SCADA), entornos de contabilidad/finanzas, o incluso blockchains híbridos. La idea recuerda la “conciencia estatal” de sistemas blockchain: un S.O. que registra cada evento sería útil donde la corrupción o ataques deben detectarse rápidamente. También podría interesar en IA/edge computing, donde se coordinan múltiples agentes y se requiere transparencia.
- **Riesgos:** Sin prueba de concepto, hay riesgos de diseño: el determinismo estricto puede llevar a latencias altas si no se maneja adecuadamente; el modelo inmutable aumenta consumo de espacio (aunque se compensa con hashing); la ausencia de pruebas deja duda sobre su factibilidad real. Asimismo, al no definir gran parte del stack, no se conocen amenazas específicas (e.g. ataques DoS al mempool).

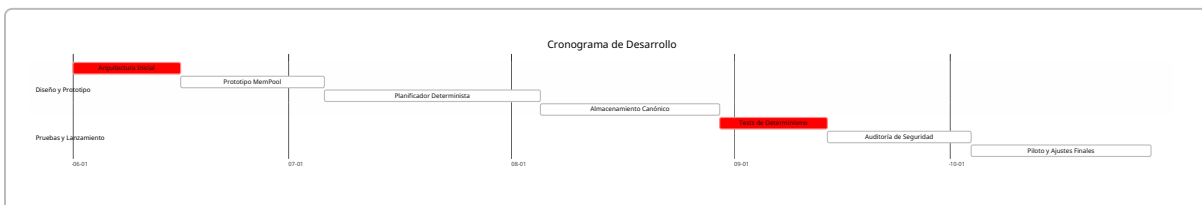
Referencias Externas y Proyectos Relacionados

- **Mempool en Bitcoin:** Entendemos el mempool como en Bitcoin: “*el conjunto de transacciones pendientes y no confirmadas*” en cada nodo ¹. Trabajos recientes estudian mejoras del mempool para eficiencia o resistencia a ataques (p.ej. *cluster mempool* ⁹ o *LØ* para mitigar MEV ⁴). Estas referencias demuestran interés académico en gestionar mempools de manera segura y determinista.
- **Sistemas deterministas:** La literatura de S.O. tiempo-real y sistemas deterministas destaca la importancia de planificación fija, bajos tiempos de respuesta y pools de memoria fijos ² ⁷. Páginas como la de INCIBE definen el determinismo temporal de tareas ². En el ámbito blockchain, el estado canónico y la determinación del próximo estado son claves ⁸, lo cual conecta con el enfoque propuesto.
- **Almacenamiento canónico:** Proyectos de content addressing (IPLD, IPFS) proporcionan modelos de datos canónicos ³. Estos sistemas descentralizados permiten referenciar datos por contenido, lo que encaja con la idea de “almacenamiento inmutable” en el S.O. propuesto.
- **Proyectos afines:** Además de *LØ* ⁴, existen sistemas operativos distribuidos que usan conceptos similares (por ejemplo sistemas basados en ledger o archivos en DAG). Sin embargo, no hay un SO directamente equivalente. El repositorio RVM Hypervisor Core (RuVix) es un esfuerzo de OS determinista en Rust, pero orientado a hiper-visores, que sirve como modelo de investigación (ver investigación RVM/RuVector).

Recomendaciones y Próximos Pasos

Para avanzar, proponemos las siguientes acciones, con esfuerzo estimado y riesgos asociados:

Acción	Esfuerzo	Riesgo
Especificar arquitectura detallada: definir módulos y flujos (crear diagramas UML/ARC)	Medio	Medio (si falta claridad inicial)
Implementar prototipo básico del mempool: estructura de datos + validación transacciones	Medio	Bajo (prueba de concepto aislada)
Desarrollar planificador determinista: algoritmo de scheduling temporal fijo	Alto	Alto (complejidad algorítmica)
Integrar almacenamiento inmutable: usar librería DAG/ Merkle (IPFS, etc.)	Medio	Medio (adaptación de herramientas)
Crear tests de determinismo: validar que ejecuciones repetidas produzcan mismos resultados	Bajo	Bajo (testeo fundamental)
Realizar auditoría de seguridad: analizar vectores de ataque al mempool y comunicaciones	Alto	Alto (fundamental para confianza)
Documentar interfaces y protocolos: definir API client/ servidor y formatos de datos	Medio	Medio (necesario para colaboración)



Resumen: El proyecto actual carece de implementación práctica, por lo que el paso inmediato es diseñar concretamente los componentes e interfaces. A corto plazo (esfuerzo bajo/medio) se recomiendan prototipos unitarios (mempool sencillo, ejecutor simulado). A mediano plazo, implementar el planificador y el almacenamiento canónico será clave (alto esfuerzo). Finalmente, pruebas extensivas y revisión de seguridad son vitales para validar el modelo. Este plan escalonado facilitará iterar rápidamente y mitigar riesgos.

Fuentes: definición de mempool ¹, S.O. determinista ², determinismo en RTOS ⁷, modelo de datos canónico ³, y casos de mempools avanzados ⁴. Estos recursos ilustran los conceptos fundamentales detrás de este sistema propuesto.

¹ An Introduction to the Mempool: What Is It And How To Explore It

<https://www.blink.sv/blog/an-introduction-to-the-mempool-what-is-it-and-how-to-explore-it>

2 Sistemas operativos de tiempo real, bastionado y funcionamiento | INCIBE-CERT | INCIBE

<https://www.incibe.es/incibe-cert/blog/sistemas-operativos-tiempo-real-bastionado-y-funcionamiento>

3 IPLD - The data model of the content-addressable web

<https://ipld.io/>

4 [2307.02081] LØ: An Accountable Mempool for MEV Resistance

<https://arxiv.org/abs/2307.02081>

5 9 Cluster mempool definitions & theory - Implementation - Delving Bitcoin

<https://delvingbitcoin.org/t/cluster-mempool-definitions-theory/202>

6 What Is Deterministic Computing? | Concurrent Real-Time

<https://concurrent-rt.com/redhawk/what-is-deterministic-computing/>

7 Theory of Operation

https://arm-software.github.io/CMSIS_5/RTOS2/html/theory_of_operation.html

8 What Is a State Machine in Blockchain Execution? | Cube Exchange

<https://www.cube.exchange/what-is/state-machine>